



Hopter: a Safe, Robust, and Responsive Embedded Operating System

Zhiyao Ma
zhiyao.ma@yale.edu
Yale University
New Haven, CT, USA

Guojun Chen
guojun.chen@yale.edu
Yale University
New Haven, CT, USA

Zhuo Chen
zhuo.chen.zc384@yale.edu
Yale University
New Haven, CT, USA

Lin Zhong
lin.zhong@yale.edu
Yale University
New Haven, CT, USA

Abstract

Microcontroller-based embedded systems are vulnerable to memory safety errors and must be robust and responsive because they are often used in unmanned and mission-critical scenarios. The Rust programming language offers an appealing compile-time solution for memory safety but leaves stack overflows unresolved and foils zero-latency interrupt handling. We present Hopter, a Rust-based embedded operating system (OS) that provides memory safety, system robustness, and interrupt responsiveness to embedded systems while requiring minimal application cooperation. Hopter executes Rust code under a novel finite-stack semantics that converts stack overflows into Rust panics, enabling recovery from fatal errors through stack unwinding and restart. Hopter also employs a novel mechanism called soft-locks so that the OS never disables interrupts. We compare Hopter with other well-known embedded OSes using controlled workloads and report our experience using Hopter to develop a flight control system for a miniature drone and a gateway system for Internet of Things (IoT). We demonstrate that Hopter is well-suited for resource-constrained microcontrollers and supports error recovery for real-time workloads.

CCS Concepts

• **Computer systems organization** → **Embedded systems; Reliability; Real-time operating systems.**

Keywords

Embedded System, Operating System, Rust, Memory Safety, Robustness, Interrupt Responsiveness

ACM Reference Format:

Zhiyao Ma, Guojun Chen, Zhuo Chen, and Lin Zhong. 2025. Hopter: a Safe, Robust, and Responsive Embedded Operating System. In *The 23rd Annual International Conference on Mobile Systems, Applications and Services (MobiSys '25)*, June 23–27, 2025, Anaheim, CA, USA. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3711875.3729149>

1 Introduction

Embedded systems have seen increasing adoption in the past decade, and their number is predicted to increase even more in the coming decades [37–39]. Due to resource constraints, many embedded systems employ microcontrollers without a memory management unit

(MMU) and as a result, do not enjoy the safety associated with virtual memory. To make things worse, C, an unsafe language, has been the most common language for programming such microcontroller-based embedded systems. Consequently, these systems can suffer from memory-related security vulnerabilities that are difficult to discover, as is evident from those reported for FreeRTOS [17, 18] a decade after its release. With microcontroller-based embedded systems increasingly used in mission-critical applications, it is imperative to enhance their memory safety and system robustness without demanding more resources or sacrificing system responsiveness.

The Rust programming language offers an attractive alternative to C. It guarantees memory safety primarily through compile-time checks, incurring little runtime overhead, and has already seen adoption in operating system (OS) development [8, 13, 14, 26, 50, 56], including embedded systems [41, 55, 63].

However, the use of Rust on microcontroller-based embedded systems faces its own challenges. First, memory safety errors linger on even with safe Rust, because its semantics assumes an infinite size of function call stacks, which is especially problematic for microcontroller-based embedded systems where there is no virtual memory and only 10s to 100s KiB of SRAM are usually available. Vulnerability reports have shown the existence of stack overflows in Rust libraries, including those intended for embedded use [15, 16, 19, 22–24]. Second, Rust’s built-in exception handling mechanism, i.e., panics, requires a stack unwinder, which is usually unavailable on microcontrollers. Without a stack unwinder, panics lead to hang or reset of the application [41] or the whole system [11, 12]. Finally, language restrictions of Rust make it difficult to implement zero-latency interrupt handling, where the OS never disables interrupts to ensure timely response to events. Known solutions [47, 48, 57, 58] are infeasible with safe Rust because they struggle to pass the compile-time check. §2 elaborates on the challenges.

In this paper, we seek to answer the following question: Can we bring memory safety, system robustness, and interrupt responsiveness to a Rust-based embedded system while requiring minimal application cooperation? We present a positive answer with the Hopter embedded OS (§3). Hopter features a co-design between the OS and the implementation language to complete memory safety and achieve fatal error resilience. Hopter also brings zero-latency interrupt handling to a Rust-based system running threaded tasks, using a novel mechanism called soft-locks.

Hopter augments Rust with finite-stack semantics (FS-semantics) to achieve stack memory safety and overflow resilience (§4.1). FS-semantics treats each function call as a potential panic site. Before executing the function body, a prologue checks the remaining stack space and will raise a panic if insufficient. In case a drop handler



This work is licensed under a Creative Commons Attribution 4.0 International License. *MobiSys '25, Anaheim, CA, USA*

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1453-5/2025/06

<https://doi.org/10.1145/3711875.3729149>

or its callee overflows the stack, Hopter will temporarily extend the stack to finish the drop handler [35, 45, 67] and raise a panic afterward. This effectively unifies stack overflows with other fatal errors using Rust panics.

Hopter then reclaims the resources upon panics utilizing a customized stack unwinder, which allows Hopter to automatically restart failed tasks (§4.2). When possible, Hopter performs a concurrent restart to expedite recovery, where the restarted task instance runs concurrently with the unwinding procedure of the failed one.

Soft-locks enable zero-latency interrupt handling by serializing mutations for each OS object, avoiding the global serialization queue and therefore unsafe Rust (§4.3). When a task or interrupt handler acquires a soft-lock that is not under contention, it gains full access to the protected object. Otherwise, it gains partial access to record its intended operations on the object and the soft-lock will commit them after contention.

We implement and open-source Hopter as a Rust library crate for Arm Cortex-M0 and Cortex-M4 architectures (§5). Hopter requires a customized compiler to compile the system, but the Rust syntax remains the same and the semantics compatible. Hopter also supports unmodified third-party hardware abstraction layer (HAL) crates, allowing application programmers to tap the growing Rust ecosystem. We evaluate Hopter by comparing it with two other embedded OSes, namely FreeRTOS and Tock (§6). Hopter has lower interrupt handling latency than FreeRTOS and is safer and more robust. Hopter requires less hardware resources than Tock and supports microcontrollers that lack a memory protection unit (MPU). We develop a flight control system for the Crazyflie 2.1 miniature drone and a gateway system for Internet of Things (IoT) devices. With the flight control system, Hopter exhibits a 56% increase in flash usage and proportionally 15% higher CPU load incurred together by FS-semantics, the unwinding mechanism, and soft-locks. The gateway system showcases Hopter's robustness against runtime errors, even on a Cortex-M0 CPU without MPU.

We make the following contributions:

- Finite-stack semantics for Rust to guarantee stack memory safety and overflow resilience, implemented via compile-time instrumentation and OS support.
- Soft-locks, a novel synchronization primitive which enables zero-latency interrupt handling on Rust-based systems with threaded tasks.
- Open-source implementation of the Hopter embedded OS [25], which integrates FS-semantics and soft-locks to deliver memory safety, failure resilience, and responsiveness, while requiring minimal application cooperation.
- Evaluation of the code size and performance overhead incurred by FS-semantics, the unwinding mechanism, and soft-locks, showing Hopter's suitability for resource-constrained microcontrollers and real-time workloads.

2 Background

Microcontrollers are widely used in embedded systems, from simple home appliances [34, 49, 65] to complex automotive control systems [28, 31, 40] and industrial automation [10, 46]. Many of these systems are unmanned or mission-critical, making robustness and the ability to recover from fatal errors essential. Due to cost

and energy constraints, the hardware resources available on microcontrollers are usually limited. They typically feature hundreds of kilobytes to a few megabytes of flash for code and read-only data, and tens to hundreds of kilobytes of SRAM for runtime data. Consequently, operating systems designed for microcontrollers must be lightweight.

Microcontrollers operate without virtual memory. All code runs in a single physical address space where flash storage, SRAM, and peripheral registers are mapped. The CPU typically executes instructions directly from the byte-addressable flash, called execute in place (XIP), while function call stacks and mutable data reside in SRAM. The code on microcontrollers usually has unrestricted access to the entire address space, making the system prone to memory safety errors like buffer overflows and invalid pointer accesses, which can lead to system instability or security vulnerabilities. Due to the lack of isolation between application tasks and the operating system, attackers can exploit memory safety vulnerabilities and easily gain full control of the entire system.

Hardware-based memory protection, e.g., memory protection unit (MPU) [2] on Arm and physical memory protection (PMP) [54] on RISC-V, is available on high-end microcontrollers and has been actively exploited by some embedded OSes, for example, Tock [41]. Unfortunately, hardware-based protection introduces overheads in code size, memory usage, and runtime performance. MPU/PMP allows developers to define up to 16 memory regions with selective read, write, or execute permissions. However, each memory region must be aligned and sized to the nearest power of two, leading to wasted flash or SRAM due to internal fragmentation. Moreover, employing an MPU or PMP requires system calls to switch privilege modes via software interrupts, and arguments need to undergo marshaling and be verified by the OS, adding performance overhead. In contrast, in systems without such protection mechanisms, system calls can be efficiently implemented as simple function calls. As a result, popular embedded OSes such as FreeRTOS [1] consider hardware-based protection optional, even if the system is written in an unsafe language like C.

Rust Programming Language is a modern systems programming language that provides memory and concurrency safety while offering direct control over hardware. It presents a promising alternative to hardware-based protection without the significant overhead incurred by managed languages. Rust has seen an increase in adoption in OS development [8, 13, 14, 26, 50, 56] and embedded systems [41, 55, 63].

Rust manages resources through its ownership model without using a garbage collector. Each value is owned by a variable, and when the variable goes out of scope, the value's *drop handler*, known as destructor function in other languages, runs to release the resource. Unlike other languages that usually make a copy or a reference when assigning a value to a new variable, Rust by default *moves* the ownership of the value from the old variable to the new one. The old variable becomes inaccessible after the move. A function call in Rust also by default moves the ownership of argument values to the callee, which is then responsible for invoking the drop handlers of these values. Consequently, the call stack is likely to reach its maximum depth while calling drop handlers, a phenomenon confirmed by our flight control system (§6.3).

Rust enforces the ownership model and analyzes reference lifetimes at compile-time to ensure memory safety, unless opted out with the `unsafe` keyword. Importantly, the compile-time check rejects some common code patterns. For instance, a struct in Rust may contain a reference only when the reference's lifetime in the syntactical scope outlives the struct's. Such restrictions preclude known implementations [47, 48, 57, 58] for zero-latency interrupt handling, where a global serialization queue is necessary to store pending operations containing references to various OS objects. To solve this problem, Hopter introduces a novel mechanism called soft-locks (§4.3) to achieve zero-latency interrupt handling.

Rust complements its compile-time check with a language exception mechanism called *panic* to forestall memory errors that are detectable only at runtime, such as out-of-bounds array access. Failed assertions through `assert!` or `unwrap` also lead to panics. A panic terminates the normal execution flow of a Rust thread. On resourceful systems such as personal computers, a panic is usually followed by a stack unwinding procedure that iterates through the function frames in the call stack and invokes the drop handlers of live objects to reclaim resources. This allows subsequent recovery from the error without restarting the entire system. However, embedded Rust systems usually lack a stack unwinder [11, 12, 41] and as a result, a panic will hang or reset the system. Worse, assertions are common in embedded Rust code [30, 41]. For system robustness, Hopter incorporates a stack unwinder optimized for microcontrollers (§4.2).

A loophole of Rust's memory safety guarantee is that function call stacks can still overflow. Rust semantics assumes an infinite stack space for each running thread, which is particularly unrealistic on microcontrollers. Common approaches to detecting stack overflows, including stack protectors (canaries) [3, 32] and periodic stack pointer inspection [4], are belated efforts. By the time of detection, the system memory is already corrupted. Prior Rust-based embedded systems either use MPU/PMP to forestall stack overflows [41], accepting the associated overhead, or employ only a single down-growing stack placed at the lower boundary of the SRAM region [11, 12], which restricts scheduling patterns. Hopter addresses this problem by running Rust code with finite-stack semantics (§4.1), which not only prevents stack overflows but also converts such errors into Rust panics to allow a unified recovery procedure through unwinding and restarting.

3 System Development with Hopter

Before we present the design (§4) and implementation (§5) of Hopter, we describe how a system developer may use it to develop an embedded system with application code.

3.1 Development Model

Hopter is open-source and a system developer receives it as a Rust library crate. Since Hopter supports threaded tasks and forsakes hardware-based protection, it can interoperate with third-party HAL libraries [29, 30], which significantly lowers the development effort. The developer must write application code in Rust and use a customized Rust compiler supplied by Hopter to build their system that uses Hopter as a dependency. Downloading and registering the customized compiler with cargo requires only three commands.

```
#[main]
fn main() {
    // Initialize resource for another task.
    // Stored behind atomic reference counting so
    // that the variable has the `Clone` trait.
    let res = Arc::new(init_resource());

    // The entry closure then also has the `Clone` trait.
    // Can spawn the task as a restartable one.
    task::build()
        .set_entry(move || another_task_main(res))
        .set_priority(8)
        .set_stack_limit(1024)
        .spawn_restartable()
        .unwrap();
}
```

Listing 1: Application code starts the execution from its main function marked by the `#[main]` attribute. Other tasks can be started dynamically through the task builder pattern. When the entry closure has the `Clone` trait, the task can be spawned as a restartable one.

The developer compiles the system with `cargo build` as usual and the Rust syntax remains the same.

Hopter expects application code to be benign, because application code may use unsafe Rust that potentially introduces memory errors. Hopter guarantees memory safety of the system as long as all unsafe code used by the application is sound. This expectation is similar to that of the popular FreeRTOS [1]; we consider it reasonable since all source code to be compiled is usually available to the system developer of a microcontroller-based embedded systems. Hopter's threat model is weaker than that of Tock [41] where application code can be malicious and the OS must isolate itself using hardware-based protection (See §2), along with its overhead.

3.2 Using Hopter Abstractions

Hopter provides three important abstractions for system developers to achieve memory safety, fault tolerance, and interrupt handling promptness. To implement application logic, a developer uses either the *restartable task* or the *fallible interrupt handler* context abstraction, which guarantees memory safety and allows recovery from fatal errors. Fallible interrupt handlers have zero-latency in response time to hardware events and can coordinate with tasks through provided *synchronization primitives*. Here we elaborate each of the three key abstractions: its benefits and how to use it. Their design and implementation details will be presented in §4 and §5, respectively.

Restartable Tasks: The tasks running on Hopter are thread-based and scheduled preemptively. They improve applications' resilience against fatal errors through automatic restarting. Applications on Hopter spawn tasks with the builder pattern as shown in Listing 1, providing the entry closure and specifying the attributes. The main task is an exception that starts execution with the function marked with the provided `#[main]` attribute, which usually initializes the system and spawns other tasks. To enable automatic restarting, the application task needs only ensure that its entry closure implements the `Clone` trait and is started with the `spawn_restartable()` method. The `Clone` trait allows Hopter to duplicate the entry environment

```
// Initialized to `Some(_)` after booting.
// Spin lock used to satisfy interior mutability rule.
static TIMER: Spin<Option<CounterUs<TIM2>>>
    = Spin::new(None);
static MAILBOX: Mailbox = Mailbox::new();

// Invoked periodically by the timer IRQ.
#[handler(TIM2)]
fn tim2_handler() {
    TIMER.lock()
        .as_mut().unwrap() // Unwrap `Option`
        .wait().unwrap(); // Acknowledge IRQ
    // Resume the task.
    MAILBOX.notify_allow_isr();
}

// Spawned as a task.
fn another_task_main() {
    loop {
        MAILBOX.wait(); // Block and yield CPU
        // Do something else ...
    }
}
```

Listing 2: Synchronization between a handler and task using a mailbox. The interrupt handler is declared through the `#[handler(...)]` macro. The handler has zero-latency response time while still being able to call synchronization primitive methods. Peripheral access is provided through a third-party HAL crate [30].

necessary to restart a task. A closure has the `Clone` trait if all of the enclosed variables are `Clone`. Upon fatal errors like stack overflows, out-of-bounds array accesses, and failed assertions, restartable tasks terminate, release their resources, and automatically restart execution from beginning. Restartable tasks do not revert to an explicit checkpoint. Instead, values stored in the heap or declared with static persist across task restarts, while function local values are dropped. If the entry closure is not `Clone`, the `spawn()` method is still available to run the task, but the task will only terminate with resources released upon fatal errors rather than restart.

Fallible Interrupt Handlers: The interrupt handlers on Hopter are functions that respond to hardware interrupts (IRQ), sharing a single interrupt stack and always preempting tasks or lower-priority handlers. They improve system robustness by tolerating fatal errors during interrupt handling. Applications declare interrupt handler functions using the `#[handler(IRQ_NAME)]` attribute macro as shown in Listing 2. A handler terminates upon fatal errors with resources released, and will rerun if the interrupt is still pending. Hopter cannot recover from interrupt stack overflows, due to the restrictions of dynamic memory allocation within the interrupt context.

Synchronization Primitives: Hopter provides applications with synchronization primitives in Rust struct types to facilitate inter-context coordination. They allow synchronization between interrupt handlers and tasks without compromising the zero-latency response time to interrupts. Application code interacts with synchronization primitives by calling defined methods. Listing 2 shows an example of the synchronization between a timer interrupt handler and a task using a mailbox. The hardware timer is accessed through a third-party HAL crate [30].

4 System Design

We next present the key design aspects of Hopter to realize memory safety, fault tolerance, and interrupt responsiveness. Hopter employs a compiler-based mechanism to guarantee memory safety for restartable tasks and fallible interrupt handlers. Notably, for stack memory safety, Hopter executes Rust code with finite-stack semantics (FS-semantics), facilitated by compile-time instrumented code (§4.1). FS-semantics unifies runtime memory errors and other fatal errors as Rust panics, allowing Hopter to apply a universal recovery mechanism based on unwinding and restarting (§4.2). Moreover, Hopter overcomes expressiveness challenges associated with Rust, achieving zero-latency interrupt handling with a novel mechanism called soft-locks (§4.3) to support the implementation of synchronization primitives. Application logic remains agnostic to both the compile-time instrumentation and OS-internal implementation, allowing existing Rust libraries to be used unmodified.

4.1 Rust with Finite-stack Semantics

Hopter executes Rust code with *finite-stack* semantics (FS-semantics) to ensure stack memory safety and allow system recovery from stack overflows. FS-semantics associates each Rust thread of execution with an implicit stack size. Hopter forestalls any function call made without sufficient free stack space by raising a panic (§4.1.1). In the exceptional case where an overflow occurs during the execution of a drop handler, Hopter temporarily extends the stack to finish the drop handler and raises the panic afterward (§4.1.2). Hopter takes two steps to achieve FS-semantics:

4.1.1 Forestalling Stack Overflows. Hopter detects an impending stack overflow by examining the free stack space before executing a function body. This is achieved by a prologue of instructions emitted by the compiler before the function body that allocates a stack frame. The prologue computes the free stack size as the difference between the current stack top indicated by the stack pointer and the stack region boundary stored in a task-local variable `BOUNDARY`. If there is insufficient free space, it traps into the OS for further diagnosis. The execution of the function prologue requires at most 8 bytes of reserved stack space on Arm.

Hopter raises a panic to the task experiencing a stack overflow. The panic initiates stack unwinding that reclaims the resources from the failed task to allow restarting it without leaked resources or deadlock. In the common case, Hopter raises a panic immediately upon detecting an imminent overflow by the prologue. After trapping, Hopter sets the program counter of the task to the stack unwinder entry to start the unwinding.

In the rare case where the stack overflows during the execution of a drop handler, Hopter extends the stack to finish running the drop logic before raising a panic. This is because stack unwinding must not start inside a drop handler, as doing so would skip some code responsible for releasing resources. More precisely, Hopter extends the stack and defers the panic if the function overflowing the stack meets either of the following two criteria: (1) Is a drop handler, or (2) Is called directly or indirectly by a drop handler.

Two separate mechanisms are required to check these criteria. The first criterion is addressed by the function prologue, which passes to Hopter whether the offending function is a drop handler.

Checking for the second criterion requires drop handler instrumentation. Whenever a drop handler starts executing, it sets the `IN_DROP` task-local variable to `true`. The drop handler resets the variable to `false` after finishing its logic. In the case of nested drop handler invocations that occur with nested struct types, only the outermost one clears the flag before returning. Hopter can determine if the second criterion is met by observing the value of the `IN_DROP` flag. Note that the function prologue is still applied on top of the instrumentation for drop handlers.

Hopter defers raising the panic if either of the criteria holds upon overflow. It sets the `OVERFLOWED` task-local variable to `true` and extends the stack, rather than raising a panic immediately. After finishing the drop logic, the outermost drop function will check the `OVERFLOWED` flag and raise the deferred panic if necessary.

Since any function can overflow the stack and initiate stack unwinding under FS-semantics, this conflicts with some compiler optimizations. The LLVM compiler backend infers the `nounwind` attribute for functions and simplifies generated code based on it. LLVM marks a function as `nounwind` if it satisfies the following two conditions [42, 43]: (1) The function body contains no side-effect instruction. (2) The function makes calls to only `nounwind` functions or is a leaf function. LLVM then simplifies the code by omitting the landing pads and unwind table entries if a call is made to a `nounwind` function. However, if such a function call overflows the stack, the stack unwinding will fail, causing a hang or system reset. Therefore, Hopter disables these compiler optimizations for correctness.

4.1.2 Extending Stacks. Hopter leverages segmented stacks [35, 45, 67] to extend stacks on systems without virtual memory. A segmented stack is a linked list of non-contiguous memory chunks called stacklets. It dynamically allocates a new stacklet upon function calls if the remaining stack space is insufficient, and frees it upon function return.

The function prologue (§4.1.1) facilitates stack extension by providing Hopter with the requested stack frame size and stack-passed argument size. These two numbers are constants known to the compiler during compilation and are embedded in the instruction sequence as literals to avoid using extra registers. Hopter subsequently fetches the values by following the program counter prior to the trap and then allocates a stacklet with a size no less than the sum of the two numbers. If stack-passed arguments exist, Hopter will also copy them to the newly allocated stacklet so that they are adjacent to the callee’s stack frame. Finally, Hopter changes the task’s stack to the new stacklet and resumes the task so that the latter continues to execute the function body.

We note that the stack extension mechanism requires software-based overflow detection. This is because the stack change must happen before executing the function body. In contrast, hardware-based protection typically detects the overflow after the function body has started execution and accessed an invalid memory address. Transferring execution to a new stacklet midway is very difficult because copying a stack frame breaks internal references.

4.2 Recovery from Panics

Hopter supports recovery from Rust panics in both tasks and interrupt handlers. A panic occurs due to an out-of-bounds array access, a failed assertion, or a stack overflow under FS-semantics.

When a task or an interrupt handler panics, Hopter terminates its execution and reclaims allocated resources. The panic is isolated within the context where it is raised and does not prevent the system from continuing execution. If a task’s entry closure implements the `Clone` trait, Hopter can automatically restart it upon a panic. Thus, interrupt handlers on Hopter are *fallible* and tasks *restartable*.

4.2.1 Unwinding Stacks. Hopter integrates a stack unwinder to reclaim resources upon panics, enabling graceful termination of tasks or interrupt handlers and allowing subsequent recovery. The unwinding procedure runs in the context of the panicked task or interrupt handler, during which the scheduler can continue to perform context switches and higher-priority interrupts can nest atop. Logically, the unwinder forces the immediate return of active functions when a panic occurs, starting from the top of the call stack and proceeding until the entry function of a task or interrupt handler returns. Local objects are dropped during this procedure. Mechanically, the unwinder refers to the unwind table generated by the compiler to restore registers and invoke small pieces of code called landing pads. There are two types of landing pads: `cleanup` pads, which call drop handlers, and `catch` pads, which terminate the unwinding procedure.

Hopter’s stack unwinder differs from existing ones [36, 53, 64] in two ways to support segmented stacks. First, Hopter’s unwinder avoids making divergent function calls that never return. Since the unwinder may be invoked to clean up a task experiencing a stack overflow, it must extend the stack to execute its own logic. If the unwinder made divergent calls, the stacklets used would not be freed, leading to memory leaks. In contrast, prior implementations typically make divergent calls to landing pads. Second, Hopter’s unwinder recognizes stacklet boundaries and frees stacklets during unwinding to prevent memory leaks. This is important because a task’s stack may contain multiple stacklets, e.g., when an overflowing drop handler raises a deferred panic while running on an extended stacklet.

4.2.2 Restarting Applications. With resources reclaimed by the unwinder, Hopter can recover a panicked task by restarting it from its entry closure. To enable restarting, Hopter requires the task’s entry closure to implement the `Clone` trait so that it can safely duplicate the closure’s enclosed environment. For interrupt handlers, Hopter performs an exception return instead of re-executing the handler after catching the panic. If the interrupt request is still pending, the handler will automatically be invoked again.

To speed up recovery from task panics, Hopter implements the *concurrent task restart* optimization proposed by [44]. This technique allows for immediate execution of a new instance of the panicked task, running concurrently with the unwinding procedure of the previous instance, thereby ensuring minimal unresponsive time. Hopter reduces the task priority of the panicked instance to the lowest, allowing unwinding to utilize idle CPU time. Rust’s ownership model eliminates race conditions between the restarted instance and the one being unwound. Application programmers may opt out of concurrent restart to prevent the restarted task from observing logically inconsistent data. In this case, Hopter performs a *context reuse* optimization, reusing the task structure of the unwound task for the restarted instance.

4.3 Zero-latency Interrupt Handling

Hopter supports zero-latency interrupt handling, invoking the handler function immediately upon receiving an interrupt. This is possible because Hopter never disables interrupts. To prevent race conditions between the interrupt handler and the preempted context, Hopter adopts a novel mechanism called *soft-locks* that are amenable to Rust's compile-time check. These locks protect OS data structures by serializing mutations from concurrent contexts, allowing interrupt handlers in Hopter to interact with OS objects, e.g., using synchronization primitives to notify tasks.

4.3.1 Algorithm Description. Soft-locks eliminate race conditions caused by concurrent access from interrupt handlers. Acquiring a soft-lock yields either *full access* or *partial access* to the protected data structure. Code gets full access if the soft-lock is not already acquired or otherwise partial access. Full access allows mutations to all data structure fields, while partial access permits modifications to only a subset of fields.

An interrupt handler receives partial access to record its intended operations when a preempted context holds the full access. These operations are deferred until the handler returns and the code with full access completes its own operation. To further prevent race conditions arising from concurrent task execution, the scheduler is always suspended before acquiring a soft-lock, thus code running within task contexts always receives full access. Because interrupt handlers always preempt tasks and have strict priority, code with full access resumes execution after the handler with partial access finishes. Therefore, deferred operations can run without conflict immediately after the code with full access finishes its logic.

Soft-locks encapsulate the protected data structure and maintain two atomic boolean flags to track contention states and whether any deferred operations are pending. Listing 3 shows the pseudo-code for acquiring and releasing soft-locks. The `locked` flag indicates whether the data structure is under contention, and the `pend` flag indicates if there are deferred operations. The protected data structure must implement the `AllowDeferredOp` trait to specify which fields are accessible through the full or partial accessor and the code for running deferred operations. Upon releasing a soft-lock, pending deferred operations are checked and executed if necessary using the scoped full accessor, which makes it amenable to compile-time checks. In the actual implementation, soft-locks are released automatically through the drop functions of access guards instead of manually.

4.3.2 Use Cases in Hopter. Soft-locks protect four data structures in Hopter. Table 1 lists these data structures, along with the operations under full or partial access and the deferred operations. The wait queue provides the foundation to implement synchronization primitives like mutexes, condition variables, semaphores, and channels. The ready queue maintains ready tasks for the scheduler. The mailbox is a lightweight synchronization primitive that allows tasks to wait for a notification, optionally with a timeout. The sleep queue stores sleeping tasks waiting for virtual timers to expire. An interrupt handler may notify a task by removing it from the wait queue or mailbox, adding it to the ready queue, and optionally deleting it from the sleep queue.

```
struct SoftLock<T> where T: AllowDeferredOp {
    locked: AtomicBool, pend: AtomicBool, ...
}

fn acquire(sl: &SoftLock<T>) -> Access {
    let prev_locked = sl.locked.swap(true);
    if prev_locked { return Access::Partial; }
    else { return Access::Full; }
}

fn release(sl: &SoftLock<T>, acc: Access) {
    match acc {
        Access::Partial => { sl.pend = true; }
        Access::Full => {
            loop {
                let prev_pend = sl.pend.swap(false);
                if prev_pend { acc.deferred_op(); }
                sl.locked = false;
                if !sl.pend { break; }
                else { sl.locked = true; }
            }
        }
    }
}
```

Listing 3: Pseudo-code for the acquire and release operation on soft-locks. Protected data structures must implement the `AllowDeferredOp` trait to specify what fields are accessible through the `Full` or `Partial` accessor and what code to execute when running `deferred_op()`. Soft-locks return the `Full` accessor under no contention or otherwise `Partial`. Deferred operations are executed when releasing with the `Full` accessor. The loop avoids missing deferred operations that come after the first check of the `pend` flag.

When a protected data structure is under contention, the handler uses partial access to record its intended operations, which are deferred for later execution. For example, consider a task that blocks on a mailbox to wait for a hardware event. The task first acquires full access to the mailbox to enter the placeholder H . If the interrupt occurs during the modification of H , the handler will get partial access to the mailbox that only allows it to increment the notification counter x . After the handler returns, the task resumes its operation, but will then notice the incremented counter when releasing the full access. The task then decrements the counter and cancels its blocking. In a different scenario where the interrupt occurs after the task finishes its operation, the task will have already released the full access and blocked itself. The handler will acquire full access to the mailbox and unblock the task in H .

5 Implementation

We have implemented Hopter for Arm Cortex-M4 and Cortex-M0 architectures with approximately 15,000 lines of code. To use Hopter for a system with a microcontroller of either architecture, a developer only needs to define an interrupt vector array, because Hopter leverages existing HAL libraries for peripheral access. The implementation consists of two parts: compiler modifications and Rust code for the library crate. Additionally, we implement Hopter's features in a hierarchy so that developers can make trade-offs between their benefits and overheads.

Table 1: OS data structures protected by soft-locks. Code gets full access to the data structure to perform one of the listed operations if the soft-lock is not under contention. Code running in task context always gets full access. An interrupt handler gets partial access if the data structure is being accessed in the preempted context, in which case the handler can access only the underlined fields to record its intended operation. Deferred operations run immediately before the code releases full access.

Data structure	Fields	Full access	Partial access	Deferred operation
Wait queue	Linked list <u>L</u> Counter <u>x</u>	1. Add a task to L 2. Remove the highest priority task from L	Increment <u>x</u>	Remove <u>x</u> high-priority tasks from L and clear <u>x</u>
Ready queue	Linked list <u>L</u> Lock-free buffer <u>I</u>	1. Add a task to L 2. Remove the highest priority task from L	Insert a task to <u>I</u>	Add tasks in <u>I</u> to L and clear <u>I</u>
Mailbox	Task placeholder H Counter <u>x</u>	1. Set H to hold a task or decrement <u>x</u> 2. Take the task from H or increment <u>x</u>	Increment <u>x</u>	Clear H and decrement <u>x</u> if H is not empty
Sleep queue	Linked list <u>L</u> Lock-free buffer <u>D</u>	1. Add a task to L 2. Remove expired tasks from L	Insert a task to <u>D</u>	Remove expired tasks and tasks in <u>D</u> from L, then clear <u>D</u>

Most of the implementation is architecture-independent. Only two small parts are not: the modification to the compiler to generate the function prologue and the inline assemblies in the library crate. Although all instructions supported by Cortex-M0 are also legal on M4, we use some M4-only instructions to reduce code size and improve performance on M4. Because M0 lacks atomic instructions, such as `ldrex` and `strex`, we implement atomic update operations for Cortex-M0 using global interrupt masking.

Compiler Modification. We modified both the `rustc` compiler frontend and the LLVM backend to support FS-semantics, and we have open-sourced the patches. To generate the function prologue (§4.1.1), we added the `split-stack` attribute to all functions during the intermediate representation (IR) emission stage in `rustc`. This attribute triggers a prologue adjustment step in LLVM’s frame lowering stage, which we further modified to produce our desired prologue. To instrument drop handlers (§4.1.1), we modified the drop elaboration step in the mid-level IR (MIR) stage of `rustc`. In total, we added 260 lines of code to the frontend and 270 lines to the backend. We also removed 30 and 110 lines from the frontend and backend, respectively, to prevent optimizations based on the `nounwind` attribute (§4.1.1). Finally, we modified 50 lines in Rust’s core library to strip away unused debug information fields in `PanicInfo`, which can reduce the compiled binary size by a few kilobytes.

Library Crate. We implemented Hopter as a Rust library crate consisting of approximately 12,200 lines of Rust code, of which fewer than 1,000 are unsafe Rust. Hopter resorts to unsafe Rust code only when certain functionalities cannot be expressed in safe Rust, such as stack extension and dynamic memory management. The unsafe code also includes inline assemblies for direct register access, bootstrap and interrupt handler entry points, and primitive memory operations such as `memset`. We also implemented two auxiliary library crates: one to provide Hopter with the `#[main]` and `#[handler]` attribute macros, and the other to allow applications to configure OS parameters. These consist of approximately 1,000 and 40 lines of safe Rust, respectively. In addition, Hopter depends on open-source Rust library crates for implementing spin locks [66], intrusive data structures [21], lock-free data structures [20], and accessing CPU core peripherals [33]. Although these crates contain unsafe code within their provided safe abstractions, they have been

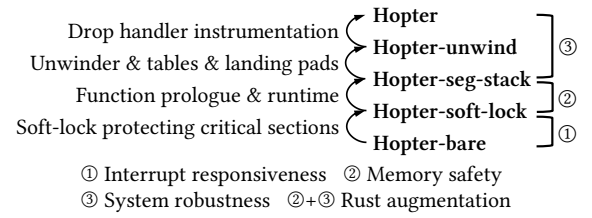


Figure 1: A breakdown of Hopter’s unique features. The bottom “bare” version includes none of the components while the top includes all. Each version in the hierarchy has one more feature included compared to the one below it.

widely used and scrutinized by the Rust community with at least millions of downloads of each.

Feature Hierarchy. We made each Hopter’s feature optional in a hierarchical manner, as shown in Figure 1. The hierarchy starts from the “bare” version that includes none of the features. Continuing up, the “soft-lock” version enables zero-latency interrupt handling by substituting soft-locks for interrupt masking in critical sections. The “seg-stack” version ensures stack memory safety and allows stack extension by generating prologues for each compiled function and incorporating a stack extension runtime. The “unwind” version supports resource reclamation upon Rust panic by incorporating the customized stack unwinder. The compiler also generates unwind tables and landing pads to assist the unwinder. Finally, the full Hopter version enables system resilience against stack overflows by instrumenting drop handlers and disabling compiler optimizations based on the `nounwind` attribute.

Atomic Operations for Cortex-M0. Because M0 does not support atomic instructions, particularly `ldrex` and `strex`, we implement atomic update methods on Cortex-M0 by globally disabling interrupts as Hopter needs these methods to implement soft-locks and to use the standard reference counting type `Arc`. These are compare-and-swap (CAS) and fetch-update (e.g., `fetch_add`), which are defined as methods on atomic types, such as `AtomicBool` and `AtomicPtr`, and atomic integer types, such as `AtomicU32`. Listing 4 shows the pseudo-code for the CAS logic. The code for the fetch-update operations is similar. Overall, this adds around 900 lines of code to the Rust core library.

```

fn compare_and_exchange(
  x: &mut AtomicU32, expect: u32, new: u32
) -> (bool, u32) {
  // Disable interrupts
  let irq_disabled = is_irq_disabled();
  disable_irq();

  // CAS logic
  let cur = x.val; let succ = false;
  if expect == cur { x.val = new; succ = true; }

  // Enable interrupts
  if !irq_disabled { enable_irq(); }
  return (succ, cur);
}

```

Listing 4: Pseudo-code for the atomic compare and exchange operation implemented for Cortex-M0. The operation compares the atomic integer’s current value with the expected value, and if they match, store the new value. Disabling interrupts ensures the atomicity of the operation.

Implementing atomic update operations by disabling interrupts has a small impact on Hopter’s interrupt handling latency, confirmed by our measurement results (Figure 2). More importantly, interrupts are disabled for only a short constant duration while executing the atomic logic, which is less than 10 instructions. Therefore, Hopter’s interrupt handling latency is still constant bounded.

6 Evaluation

Hopter aims to provide system memory safety, robustness, and responsiveness, while requiring minimal application cooperation. Because memory safety is achieved by construction, this section quantifies robustness and responsiveness, as well as the overhead brought by the unique design choices of Hopter. Specifically, we seek to answer the following questions:

- (Q1 Overhead): What is the overhead introduced by Hopter in order to achieve its objectives?
- (Q2 Size): Is Hopter light enough to support microcontrollers with small flash and SRAM?
- (Q3 Responsiveness): Can Hopter support time-sensitive tasks and interrupt handling?
- (Q4 Robustness): Can Hopter recover from more fatal errors than other OSes and be fast enough for mission-critical applications?

6.1 Methods

We employ three orthogonal methods to answer these questions.

First, to quantify the impact by each of the features of Hopter, we follow the hierarchy in Figure 1 and measure the statistics for each listed variant.

Second, we compare Hopter with two well-known embedded OSes: FreeRTOS [1], the most widely used embedded OS, and Tock [41], the state-of-the-art Rust-based embedded OS. We develop a set of microbenchmarks with controlled workload for all three OSes. FreeRTOS employs no mechanisms to provide memory safety or robustness against application errors, and is designed to support high-frequency workloads and to be light-weight. Tock

assumes application code can be malicious and employs hardware-based protection.

Finally, in order to assess Hopter’s performance and overhead under a realistic application and demonstrate its ability to recover from errors under mission critical scenarios, we develop two embedded systems using Hopter, a flight control system for the Crazyflie [6] miniature drone and an Internet-of-Things (IoT) gateway.

6.2 Controlled Workload

We perform our micro-benchmarks with two evaluation boards: the STM32F412G-Discovery board equipped with an Arm Cortex-M4 CPU, 256 KiB SRAM, and 1 MiB flash, and the STM32F072B-Discovery board with an M0 CPU, 16 KiB SRAM, and 128 KiB flash. We configure the M4 CPU to run at 96 MHz and the M0 at 48 MHz, and we enable the floating point unit on M4. We set the compiler to optimize for speed (-O3). For our experiments, we first build an LED blinking application to show the minimum resource required when developing with an OS. Next, we quantify system responsiveness by measuring the latency of the task context switch and response to interrupts. Finally, we demonstrate that Hopter can recover from more complex scenarios than other OSes with a serial server task.

6.2.1 Minimum System (Q1/Q2). To measure the minimum hardware resources required to run a system developed using the three OSes under comparison, we build a blinking LED application, the “hello world” program for the embedded world. We assume an application programmer’s role, i.e., using the OS and hardware abstraction layer (HAL) library as-is through their public APIs, without modifying their internal implementation. We use the HAL library from stm32-rs [30] and STM32Cube [61, 62] for Hopter and FreeRTOS, respectively. Tock comes with its own HAL.

The minimum system columns in Table 2 list the flash and SRAM size. The flash overhead of Hopter mainly comes from the stack unwinder logic, the unwind table, the landing pads, and the drop handler instrumentation. A minor overhead comes from the function prologue and stack extension runtime. The flash overhead can be amortized with a more sophisticated application like the flight control system (§6.3), for which we provide a more detailed breakdown of its 56% flash overhead.

The minimum hardware resources required to run Hopter without robustness features, i.e., the “bare”, “soft-lock”, or “seg-stack” variants, are similar to those of FreeRTOS. The support for stack unwinding and the drop handler instrumentation incur noticeable overhead particularly in flash usage, but the numbers are still within the lower tens of KiB. On the other hand, systems developed with Tock requires significantly larger flash and SRAM as shown in Table 2, because Tock is compiled separately from the application. The compiler toolchain is unable to remove unused OS code at compile time, resulting in the size bloat.

6.2.2 Task Scheduling and Interrupt Handling (Q1/Q3). To compare the system responsiveness, we measure the following performance metrics: the latency of a context switch between two tasks and the latency to respond to an interrupt from a handler or task.

Context switch latency reflects the overhead for inter-task co-operation. Hopter’s latency is on the same order of magnitude as FreeRTOS’s, allowing it to support multiple cooperating tasks

Table 2: Comparing Hopter to FreeRTOS and Tock, showing the overheads of Hopter’s components, and showing the overhead of implementing atomic update operations by disabling interrupts. Hopter requires more flash than FreeRTOS mainly because it includes additional components to improve robustness. Hopter requires significantly less flash and memory than Tock that compiles the OS code separately. Hopter has the lowest and most consistent interrupt response latency, benefiting from the soft-lock. Hopter’s context switch performance and task notification latency is close to FreeRTOS’s and an order of magnitude lower than Tock’s. Most flash and performance overhead of Hopter comes from the components for robustness, while the responsiveness and safety components introduce only small overhead. Hopter provides all three features on Cortex-M0 like on M4, but the implementation of atomic update operations by disabling interrupts increases the latency of context switch and interrupt handling by 20%.

		Minimum system		Latency (μ s)			Feature		
		Flash (KiB)	†SRAM (KiB)	* Task	** Interrupt	** Interrupt-task	Responsive	Safe	Robust
OS	FreeRTOS	13.70	3.76	8.0	2.20 (0.39)	12.76 (0.73)	✓	✗	✗
	Tock	‡147.5 + 6.60	‡66.53 + 0.75	264.7	151.54 (26.03)	365.14 (26.71)	✗	✓	✓
	Hopter	27.68	1.00	16.0	1.36 (0.01)	31.88 (2.44)	✓	✓	✓
Component	Hopter-unwind	22.71	1.00	12.9	1.32 (0.03)	26.44 (1.96)	✓	✓	✗
	Hopter-seg-stack	12.60	0.84	13.0	1.20 (0.01)	27.24 (1.90)	✓	✓	✗
	Hopter-soft-lock	10.05	0.75	12.6	1.16 (0.02)	25.48 (1.84)	✓	✗	✗
	Hopter-bare	9.12	0.49	15.5	5.26 (1.21)	28.30 (1.83)	✗	✗	✗
Architecture	FreeRTOS (M0)	12.36	3.75	19.2	4.88 (1.15)	29.50 (4.43)	✓	✗	✗
	FreeRTOS (M0 on M4)	13.32	3.75	7.3	2.12 (0.43)	12.22 (1.66)	✓	✗	✗
	Hopter (M0)	25.60	0.92	48.7	3.18 (0.05)	76.20 (7.03)	✓	✓	✓
	Hopter (M0 on M4)	25.67	0.92	19.9	1.64 (0.02)	38.64 (2.83)	✓	✓	✓

†: Excluding function call stacks, whose sizes are configurable. ‡: OS plus application size.

*: Average from 10,000 trials. **: Maximum and standard deviation from 10,000 trials.

M0: Measured on STM32F072B-Discovery (Cortex-M0). Others are measured on STM32F412G-Discovery (Cortex-M4).

M0 on M4: Code is compiled with only M0 instructions but runs on STM32F412G-Discovery (M4).

with frequencies up to 1,000 Hz (§6.3). Tock on the other hand is substantially slower, because each context switch from a task to another requires four context switches between the OS and applications, rendering Tock not suitable for performance-demanding workloads. We compute context switch latency by running two tasks performing context switches to each other using the most efficient synchronization primitive available, i.e., mailbox on Hopter, task notification on FreeRTOS, and inter-process communication (IPC) on Tock. The task column in Table 2 lists the average latency by measuring 10,000 context switches using a hardware timer on the microcontroller precise to 1 μ s. Hopter’s latency is higher than FreeRTOS’s mainly due to the use of safe Rust in implementing task lists and bookkeeping the current running task. The optimized organization and operations of task list found in FreeRTOS are incompatible with Rust’s ownership model (see §2). However, an 8 μ s overhead in context switch results in less than 1% CPU load increase for a task running at 1,000 Hz.

We also measure the interrupt response latency, which determines if the system may miss an ephemeral event. The measured board registers an interrupt handler to be triggered by a rising edge on a general-purpose input/output (GPIO) pin. The handler simply sets another GPIO pin to high to respond, either directly or by notifying a high-priority task to do so. We program another board to trigger the rising edge and measure the response latency with precision of 0.02 μ s.

The Interrupt column in Table 2 reports the maximum latency observed in 10,000 tests in which the interrupt handler directly responds to the interrupt, while the Interrupt-task column reports that observed when the handler notifies a high-priority task to respond. Since Tock does not support declaring an application interrupt handler, we instead register the handler as a capsule in its

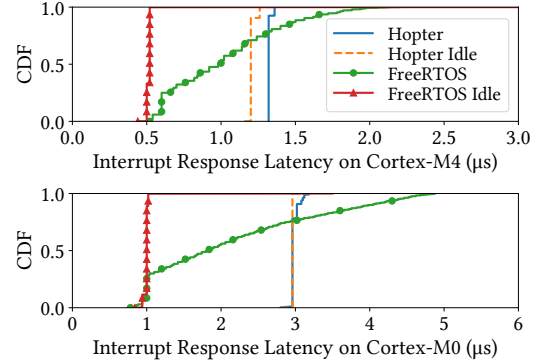


Figure 2: Hopter’s interrupt response latency is almost constant using soft-lock on Cortex-M4. Background workload causes slight increase in latency due to the stress on instruction and literal cache. Hopter’s latency on Cortex-M0 has small variance due to how it implements atomic operations on M0, namely disabling interrupts. In contrast, FreeRTOS’s latency is sensitive to background workload, because it masks interrupts inside its critical sections.

kernel space. The latency numbers are obtained when two other tasks are context switching to each other as the system background workload. Hopter has the lowest and most consistent latency when responding in the interrupt handler, benefiting from the soft-lock mechanism. When handling the interrupt with a task, Hopter becomes slower than FreeRTOS due to its slower context switch. In both cases, Hopter is orders of magnitude faster than Tock.

Hopter’s soft-lock is effective in maintaining a consistent interrupt response latency regardless of the background workload, as shown in Figure 2. The slight increase in the number under load is due to the eviction of cached instructions and literals in the ART



Figure 3: Crazyflie 2.1, a commercial off-the-shelf miniature drone. We implement a flight control system with Hopter to evaluate its capability of supporting real-time applications.

accelerator for flash [59]. In contrast, FreeRTOS’s latency is sensitive to the background workload. Masking interrupts in its critical sections causes a delay in the interrupt response, and such a delay will be exacerbated if the CPU runs at a lower frequency as on M0 or the OS is under heavier load.

Hopter is responsive when running on M0, because the atomic update operations disable the interrupts for only a short constant period. The slowdown on M0 is mainly due to the CPU being less performant than M4. To measure the overhead caused by the longer function prologue and atomic update operations on M0, we run the same experiments on the M4 board but compile the code using only the instructions available on M0. Hopter’s latency of context switch and interrupt response increases by 20%. FreeRTOS is slightly faster when running the M0 code on M4 than when running the M4 code because the M0 code does not preserve and restore floating-point registers during task context switch.

6.2.3 Fatal Error Recovery (Q4). Hopter allows a faulty application task to perform arbitrary cleanup logic during resource reclamation, which enables the system to recover from more complex scenarios and overcome the limitations of a simple task restart. We implement an application as a proof-of-concept consisting of three tasks: A serial server task that transmits data received from other tasks over the serial interface, and two client tasks that periodically send data to the server task. To prevent undesirable data interleaving between sending tasks, a client task first acquires a lock on the server before sending data and releases it after finishing.

We deliberately trigger an out-of-bounds array write in one client task while it is holding the lock. In Hopter, the fault manifests itself as a Rust panic, which causes the task to be restarted. During stack unwinding, the unwinder invokes the drop handler of the lock guard object to release the lock. After restart, both client tasks proceed as normal. In contrast, on Tock, if the written address falls in the task’s allowed address range, it becomes a silent data corruption within the task. Otherwise, the Tock detects the fault and restarts the task, but the lock will not be released, subsequently causing a deadlock. Since FreeRTOS has no safety protection, the out-of-bounds write either causes a system-wide data corruption, or triggers a hardware fault that hangs or resets the system.

6.3 Drone Flight Control

We develop a flight control system with Hopter to measure its overhead in resource usage and CPU load with a realistic application, and to quantify the latency of interrupt response and task recovery demonstrating its suitability for real-time workload. The program runs with the commercially available Crazyflie 2.1 hardware platform [6] as shown in Figure 3. The drone is powered by

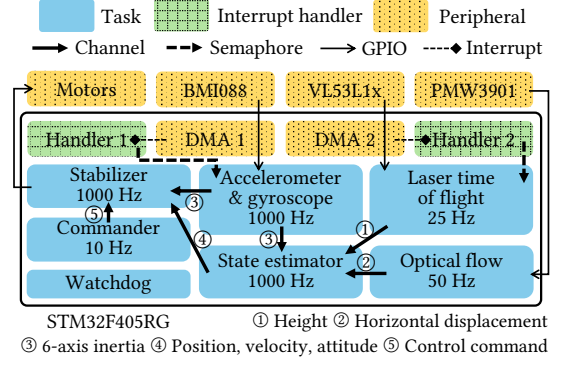


Figure 4: The flight control system consists of interrupt handlers and periodical tasks running at high frequency. Tasks and handlers synchronize through channels and semaphores. Peripherals are connected via GPIO pins.

Table 3: Binary size, SRAM usage, and CPU load of the flight control system. The SRAM usage excludes function call stacks, whose sizes are configurable. The CPU load is the average of 100 measurements when the drone is set to hover.

	Flash (KiB)	SRAM (KiB)	CPU
FreeRTOS	133.56	27.93	36.5%
Hopter	213.53	7.51	52.3%
Hopter-unwind	173.43	7.51	48.8%
Hopter-seg-stack	147.80	7.35	47.5%
Hopter-soft-lock	141.90	7.00	45.5%
Hopter-bare	136.95	6.52	48.4%

an STM32F405RG microcontroller, featuring a Cortex-M4F CPU with a maximum clock speed of 168 MHz, 192 KiB of SRAM, and 1 MiB of flash storage.

We implement the flight control system in Rust, which consists of around 10,000 lines of code. About 6,700 lines are peripheral driver code manually translated from open-source libraries [9, 51, 52]. We use the HAL library from `stm32-rs` [30] to access peripherals. The flight control system contains 22 unsafe Rust statements, mostly for interrupt configuration.

The flight control system includes seven periodical tasks that collectively manage flight control as shown in Figure 4. Three tasks are responsible for reading data from the inertial sensors, the height sensor, and the optical flow sensor, respectively. Two of them read through direct memory access (DMA). Subsequently, the State Estimator task obtains the collected data, through the data channel synchronization primitive provided by Hopter, and computes the drone’s position and attitude with the Kalman filter algorithm. Finally, the Stabilizer task utilizes the output from the estimator to modulate the power distribution across the four propeller motors, aiming to maintain the drone’s stability and follow the commands sent from the commander task. For operational safety, the Watchdog task supervises other flight control tasks, and if any of them is unresponsive for over a second, Watchdog will shut off all motors.

6.3.1 System Size and CPU Load (Q1/Q2/Q3). Table 3 shows the flash and SRAM usage of the flight control system and the CPU

load when the drone is set to hover. We break down Hopter’s flash overhead as follows, where underlined numbers are constant, while others grow with the binary size. The function prologue for stack memory safety and the stack extension runtime each incur a flash overhead of 3.14 KiB and 2.75 KiB, respectively. To enable resource reclamation through unwinding, the stack unwinder adds 9.59 KiB, the landing pads add 7.50 KiB, and the unwind table adds 8.53 KiB. To allow recovery from stack overflows, instrumenting the drop handlers increases the code size by 27.80 KiB and the unwind table by 4.70 KiB. Disabling compiler optimizations based on the `nounwind` function attribute increases the code size and the unwind table size by another 2.97 KiB and 4.64 KiB, respectively.

We also compare against a FreeRTOS-based implementation with a similar program structure. FreeRTOS’s version requires similar flash storage as with Hopter’s “bare” variant. Its higher SRAM usage is due to the use of larger static buffers, and its lower CPU load is due to the availability of highly optimized math library and the DMA driver implementation for the SPI bus to access the optical flow sensor.

Hopter’s drop handler instrumentation (§4.1.1) incurs the largest overhead in flash size and CPU load, which is counterintuitive since the instrumented code appears to be short and applies only to drop handler functions. However, we observe a cascading effect in the generated code that leads to size bloat and CPU load increase. (1) If the type of any field within a compound type implements `Drop`, the drop handler of both inner and outer types will be instrumented. (2) ARMv7, as a RISC architecture, must load the address of task local variables into registers before accessing them, resulting in longer instruction sequences. (3) There is more register spilling, further increasing the length of the instruction sequence. (4) The larger function body causes the compiler not to inline some drop handler functions, resulting in more function calls and returns. (5) An outlined function, in turn, includes additional prologue instructions.

However, drop handler instrumentation is indispensable for correct recovery from stack overflow despite the overhead. We perform a static stack depth analysis for the seven flight control tasks, ignoring indirect function calls through function pointers or trait objects (0.1% of all calls). Five tasks reach their respective maximum stack size inside a drop function. Without the instrumentation, initiating stack unwinding inside a drop handler will cause a system reset.

6.3.2 In-flight Interrupt Response Latency (Q3). We measure the interrupt response latency while setting the drone to hover, following the same experimental setup as in §6.2.2. With soft-lock, the maximum interrupt response latency over 10,000 measurements is 1.04 μ s with a standard deviation of 0.02 μ s. However, without soft-lock, the maximum latency rises to 7.56 μ s with a standard deviation of 0.57 μ s, due to the critical sections that prevent the system from responding to interrupts for an extensive period.

Prompt interrupt response is essential to support the radio on the drone. The drone employs an nRF51822 chip that forwards received radio packets to the main microcontroller via a universal asynchronous receiver-transmitter (UART) serial interface at baud rates of up to 2 Mbps. In the Syslink protocol [7] used by the drone, the length of the packet is determined by the first four bytes, so DMA can only be initiated after these bytes are received. Without DMA, the system must respond immediately to the interrupt after

Table 4: Recovery latency and stack unwinder workload for the Stabilizer task upon fatal errors. The error is either a deliberate panic!() or a stack overflow after some function recursions. Both the task context reuse and concurrent restart optimization can speed up the recovery upon panic and stack overflow. Concurrent restart maintains a constant recovery latency regardless of the stack depth.

	Panic	OF-recur-4	OF-recur-10
Unoptimized (μ s)	197	268	402
Ctxt. reuse (μ s)	134	207	335
Concur. restart (μ s)	189	192	190
# Stack frames	6	7	13
# Landing pads	2	6	12

the UART interface receives a byte. Considering that each data byte carries an overhead of one start bit and one stop bit, an interrupt response delay over 5 μ s will cause an overrun of the receive shift register [60], resulting in data loss. Only with soft-locks can the radio module function correctly.

6.3.3 In-flight Fatal Error Tolerance (Q4). We deliberately introduce panics into the flight control system while the drone is hovering to assess Hopter’s ability to recover the system from them. The drone has no visible disturbance when we introduce a panic into an interrupt handler or to the flight control tasks except the Stabilizer task. The drone rotates around 20 degrees along the yaw axis and drops a few centimeters when the Stabilizer task panics, but can still hover after the task restart. Stabilizer is the most sensitive to fatal errors because it directly modulates the power of the motors.

We also measure the recovery latency of Stabilizer on different fatal errors. For the panic test, we place a `panic!()` statement in the code that updates the stabilizer states, simulating out-of-bounds array access or a failed assertion. For the stack overflow test, we insert a call to a recursive function that defines a local object with a drop handler, which will overflow the stack after 4 or 10 recursions. Table 4 lists the latency, as well as the number of stack frames to unwind and landing pads to invoke for each test case. The recovery latency is measured as the time elapsed between the occurrence of the error and the execution of the entry function of the restarted task instance. A stack extension incurs an additional 20 μ s delay.

The measurement result demonstrates that both the task context reuse and concurrent restart optimization can reduce recovery latency. Task context reuse outperforms concurrent restart when the stack is shallow, but concurrent restart enables faster recovery by maintaining a constant latency when the stack is deep.

In all three cases, Stabilizer can recover before the next execution interval. The observed disturbance to the drone is due to the new Stabilizer task instance not having any command to follow. The motors are kept at zero power until the next command is received.

6.4 IoT Gateway

We develop a gateway system for Internet-of-Things (IoT) devices using the STM32F072B-Discovery board to showcase Hopter’s robustness against runtime errors, even on a Cortex-M0 microcontroller without an MPU. The gateway consists of a data plane and a control plane, as shown in Figure 5. The data plane receives packets from the inbound connection through an interrupt-driven UART

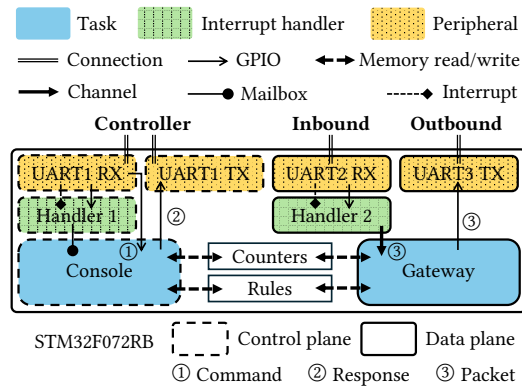


Figure 5: The gateway consists of a data plane and a control plane. The data plane forwards packets from the inbound connection to the outbound one. The control plane can gather runtime statistics and change forwarding rules. Errors in the control plane do not affect the operation of data plane thanks to Hopter’s robustness features.

interface and forwards them to the outbound UART connection. As part of the data plane, the Gateway task verifies the integrity of received packets, updates runtime counters, and optionally modifies the packets according to the rules before forwarding them. The control plane establishes a text-based control session through another UART interface. Users can send commands to query the number of forwarded and corrupted packets, count occurrences of a specific byte pattern, reset the counters, and change forwarding rules. The rules allow users to specify escaped bytes and to filter packets based on their types. As part of the control plane, the Console task parses and executes the command, and responds to the user.

The gateway uses an open-source HAL library [29], consisting of 360 lines of Rust code, with only 8 lines unsafe for configuring peripherals. The compiled binary requires 60.0 KiB of flash and about 1.3 KiB of SRAM, excluding stacks. In comparison, an implementation based on FreeRTOS requires 21.0 KiB of flash and around 3.0 KiB of SRAM.

Hopter isolates data plane connections from fatal control plane errors. To trigger an error, we make the Console task skip checking the remaining free size of the buffer when receiving commands. A long command will overflow the buffer, causing the Console task to panic and restart. Meanwhile, the data plane continues to forward packets without disruption. The packet forwarding latency is unaffected by the panic recovery procedure. In contrast, the same buffer overflow error will silently corrupt data or crash a FreeRTOS-based system. Tock does not support this board due to the lack of an MPU.

7 Related Work

Memory Safety with Rust: Similar to Hopter’s use of Rust, other OSes have leveraged Rust for memory safety across both the kernel and applications. Two recent works targeting PCs and servers, TheSeus [8] and RedLeaf [50], utilize Rust to achieve memory safety, yet they still depend on the memory management unit (MMU) to place a guard page to prevent stack overflows. RIOT [5], Embassy [11], and RTIC [12] embedded OS, as well as the kernel space

of Tock [41], employ Rust for memory safety on microcontrollers. RIOT supports threaded tasks written in Rust, but similarly uses MPU/PMP to set a guard region to prevent stack overflows. This approach will not allow recovery from stack overflows, as discussed in §4.1.2. Embassy, RTIC, and Tock’s kernel space employs a single down-growing stack placed at the lower boundary of the SRAM region to prevent stack overflows from corrupting memory. However, they support only restricted scheduling patterns and cannot recover from stack overflows. In contrast, Hopter achieves memory safety solely via software, supports arbitrary scheduling patterns with threaded tasks, and can recover from stack overflows.

Fatal Error Recovery: Few embedded OSEs for microcontrollers offer a recovery mechanism related to Hopter’s capability of recovering from fatal errors. Tock [41] can automatically restart a task that failed due to memory access violations. Zephyr [27] can also terminate a task upon such an error but require the application to register an error handler separately. Neither supports automatic cleanup for graceful task termination. In contrast, Hopter runs application cleanup logic (drop handlers), allowing the system to avoid subsequent errors such as deadlocks.

Zero-latency Interrupt Handling: Hopter employs soft-locks for zero-latency interrupt handling, unlike previous embedded OSes such as eCos [47], PEASE [58], PURE [57], and SMX [48], which split interrupt handling into top and bottom halves to avoid OS interrupt masking. The split design necessitates a global serialization queue for deferred operations (bottom halves) that reference OS objects, therefore requiring unsafe Rust because the compiler cannot verify the lifetime of the references. Also, Hopter’s soft-lock can improve performance by deferring operations only upon contention, which is rare, thus reducing CPU load by mostly taking the fast path.

8 Conclusion

This paper describes Hopter, a Rust-based embedded OS designed for microcontrollers. Hopter provides applications with memory safety, system robustness, and interrupt responsiveness while requiring minimal cooperation from them. Hopter forestalls stack overflows and converts such errors into Rust panics by executing Rust code under FS-semantics using compile-time code instrumentation and runtime OS support. By unifying all fatal errors as panics, Hopter performs stack unwinding to reclaim resources from failed application tasks or interrupt handlers, and can automatically restart failed tasks. To ensure timely response to interrupts, Hopter incorporates a novel mechanism called soft-locks to achieve zero-latency interrupt handling. Hopter is well-suited for resource-constrained microcontrollers and supports error recovery for real-time workloads. The mechanism for recovering from stack overflows is also applicable to processes running in a server, and soft-locks can be useful in reducing the latency to respond to process signals.

Acknowledgments

Zhiyao Ma, Guojun Chen and Lin Zhong were supported in part by National Science Foundation Award #2416594. Zhuo Chen was supported by Yale University. The authors were grateful for the useful feedback from their shepherd and reviewers.

References

- [1] Amazon.com, Inc. 2020. FreeRTOS: Real-time operating system for microcontrollers and small microprocessors. <https://www.freertos.org>. Version 10.3.1. Accessed: 2025-04-24.
- [2] Arm Limited. 2021. *ARMv7-M Architecture Reference Manual (Issue E.e)*. [https://developer.arm.com/documentation/ddi0403/ee/Chapter B3.5: Protected Memory System Architecture, PMSAv7](https://developer.arm.com/documentation/ddi0403/ee/Chapter%20B3.5%3AProtected%20Memory%20System%20Architecture,PMSAv7). Accessed: 2025-04-24.
- [3] Arm Ltd. 2023. *Arm Compiler Version 6.6.5 armclang Reference Guide*. [https://developer.arm.com/documentation/dui0774/A/Section 2.28](https://developer.arm.com/documentation/dui0774/A/Section%202.28). Accessed: 2025-04-24.
- [4] AWS open source. 2024. *FreeRTOS Documentation*. [https://www.freertos.org/Documentation/00-Overview Chapter: FreeRTOS Stack Usage and Stack Overflow Checking](https://www.freertos.org/Documentation/00-Overview%20Chapter%3AFreeRTOS%20Stack%20Usage%20and%20Stack%20Overflow%20Checking). Accessed: 2025-04-24.
- [5] Emmanuel Baccelli, Cenk Gündoğan, Oliver Hahm, Peter Kietzmann, Martine S Lenders, Hauke Petersen, Kaspar Schleiser, Thomas C Schmidt, and Matthias Wählisch. 2018. RIOT: An open source operating system for low-end embedded devices in the IoT. *IEEE Internet of Things Journal* (2018).
- [6] Bitcraze. 2020. Crazyflie: A flying open development platform. <https://www.bitcraze.io/>.
- [7] Bitcraze. 2024. *Syslink protocol version 2024.10*. <https://www.bitcraze.io/documentation/repository/crazyflie2-nrf-firmware/2024.10/protocols/syslink/>. Accessed: 2025-04-24.
- [8] Kevin Boos, Namitha Liyanage, Ramla Ijaz, and Lin Zhong. 2020. Theseus: an experiment in operating system structure and state management. In *Proc. USENIX OSDI*.
- [9] Bosch. 2024. BMI088. <https://github.com/BoschSensortec/BMI088-Sensor-API>. Version 1.9.0. Accessed: 2025-04-24.
- [10] Ching-Han Chen, Ming-Yi Lin, and Chung-Chi Liu. 2018. Edge computing gateway of the industrial internet of things using multiple collaborative microcontrollers. *IEEE Network* (2018).
- [11] Embassy Project Contributors. 2023. Embassy: The next-generation framework for embedded applications. <https://embassy.dev>. Accessed: 2025-04-24.
- [12] RTIC Contributors. 2023. RTIC: The hardware accelerated Rust RTOS. <https://rtic.rs/2/book/en/>. Accessed: 2025-04-24.
- [13] Jonathan Corbet. 2022. A first look at Rust in the 6.1 kernel. <https://lwn.net/Articles/910762/>. *Linux Weekly News* (10 2022).
- [14] Microsoft Corporation. [n. d.]. Windows Drivers in Rust. <https://github.com/microsoft/windows-drivers-rs>. Accessed: 2025-04-24.
- [15] MITRE Corporation. 2019. CVE-2019-25001: Flaw in CBOR deserializer allows stack overflow. <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2019-25001>. Accessed: 2025-04-24.
- [16] MITRE Corporation. 2020. CVE-2020-35858: Parsing a specially crafted message can result in a stack overflow. <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2020-35858>. Accessed: 2025-04-24.
- [17] MITRE Corporation. 2021. CVE-2021-31572: FreeRTOS before 10.4.3 has an integer overflow for a stream buffer. <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-31572>. Accessed: 2025-04-24.
- [18] MITRE Corporation. 2021. CVE-2021-32020: FreeRTOS before 10.4.3 has insufficient bounds checking during management of heap memory. <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-32020>. Accessed: 2025-04-24.
- [19] MITRE Corporation. 2024. CVE-2024-36760: A stack overflow vulnerability found in version 1.18.0 of rhai. <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-36760>. Accessed: 2025-04-24.
- [20] Alex Crichton, Jeehoon Kang, Aaron Turon, and Taiki Endo. 2024. Tools for concurrent programming. <https://crates.io/crates/crossbeam>. Version 0.8.4. Accessed: 2025-04-24.
- [21] Amanieu d'Antras. 2024. Intrusive collections for Rust (linked list and red-black tree). <https://crates.io/crates/intrusive-collections>. Version 0.9.7. Accessed: 2025-04-24.
- [22] The Rust Security Advisory Database. 2018. RUSTSEC-2018-0005: Uncontrolled recursion leads to abort in deserialization. <https://rustsec.org/advisories/RUSTSEC-2018-0005.html>. Accessed: 2025-04-24.
- [23] The Rust Security Advisory Database. 2023. RUSTSEC-2023-0080: Buffer overflow due to integer overflow in transpose. <https://rustsec.org/advisories/RUSTSEC-2023-0080.html>. Accessed: 2025-04-24.
- [24] The Rust Security Advisory Database. 2024. RUSTSEC-2024-0012: Stack overflow during recursive JSON parsing. <https://rustsec.org/advisories/RUSTSEC-2024-0012.html>. Accessed: 2025-04-24.
- [25] Hopter Developers. 2024. Hopter Embedded OS. <https://github.com/hopter-project/hopter>. Accessed: 2025-04-24.
- [26] Redox Developers. 2015. Redox - Your Next(gen) OS. <https://www.redox-os.org/>. Accessed: 2025-04-24.
- [27] Zephyr Developers. 2016. Zephyr Project. <https://github.com/zephyrproject-rtos/zephyr>. Accessed: 2025-04-24.
- [28] Guo Dong, Wang Hongpei, Gao Song, and Wang Jing. 2011. Study on adaptive front-lighting system of automobile based on microcontroller. In *Proc. IEEE Int. Conf. Transportation, Mechanical, and Electrical Engineering (TMEE)*.
- [29] Daniel Egger. 2021. Peripheral access API for STM32F0 series microcontrollers. <https://crates.io/crates/stm32f0xx-hal>. Version 0.18.0. Accessed: 2025-04-24.
- [30] Daniel Egger. 2024. Peripheral access API for STM32F4 series microcontrollers. <https://crates.io/crates/stm32f4xx-hal>. Version 0.22.1. Accessed: 2025-04-24.
- [31] Bill Fleming. 2011. Microcontroller units in automobiles [automotive electronics]. *IEEE Vehicular Technology Magazine* (2011).
- [32] Free Software Foundation, Inc. 2023. *GCC 15.0.0 Manual*. [https://gcc.gnu.org/onlinedocs/gcc/Section 3.12, Program Instrumentation Options](https://gcc.gnu.org/onlinedocs/gcc/Section%203.12,Program%20Instrumentation%20Options). Accessed: 2025-04-24.
- [33] Adam Greig. 2023. Low level access to Cortex-M processors. <https://crates.io/crates/cortex-m>. Version 0.7.7. Accessed: 2025-04-24.
- [34] Mehedi Hasan, Maruf Hossain Anik, and Sharnali Islam. 2018. Microcontroller based smart home system with enhanced appliance switching capacity. In *Proc. IEEE HCT Information Technology Trends (ITT)*.
- [35] Robert Hieb, R Kent Dybvig, and Carl Bruggeman. 1990. Representing control in the presence of first-class continuations. *ACM SIGPLAN Notices* (1990).
- [36] Free Software Foundation Inc. 2023. Exception handling and frame unwind runtime interface routines. <https://github.com/gcc-mirror/gcc/blob/721cdcd1ddde0738deb08895e113a8db84187a14/libgcc/unwind.inc>. Accessed: 2025-04-24.
- [37] Fortune Business Insights. 2024. Embedded Systems Market Size, Share & COVID-19 Impact Analysis, By Component, By Type, By Application, and Regional Forecast, 2023-2030. <https://www.fortunebusinessinsights.com/embedded-systems-market-108767>. Accessed: 2025-04-24.
- [38] Future Market Insights. 2024. Embedded System Market Analysis from 2024 to 2034. <https://www.futuremarketinsights.com/reports/embedded-system-market>. Accessed: 2025-04-24.
- [39] Global Market Insights. 2024. Embedded Systems Market Size - By Component, By Application, By Function & Forecast, 2024 - 2032. <https://www.gminsights.com/industry-analysis/embedded-system-market>. Accessed: 2025-04-24.
- [40] Prateek Khurana, Rajat Arora, and Manoj Kr Khurana. 2014. Microcontroller based implementation of Electronic Stability Control for automobiles. In *Proc. IEEE Int. Conf. Advances in Engineering & Technology Research*.
- [41] Amit Levy, Bradford Campbell, Branden Ghena, Daniel B Giffin, Pat Pannuto, Prabal Dutta, and Philip Lewis. 2017. Multiprogramming a 64kb computer safely and efficiently. In *Proc. ACM SOSp*.
- [42] LLVM. 2025. AttributorAttributes.cpp - Attributes for Attributor deduction. https://llvm.org/doxygen/AttributorAttributes_8cpp_source.html. Accessed: 2025-04-24.
- [43] LLVM. 2025. FunctionAttrs.cpp - Pass which marks functions attributes. https://llvm.org/doxygen/FunctionAttrs_8cpp_source.html. Accessed: 2025-04-24.
- [44] Zhiyao Ma, Guojun Chen, and Lin Zhong. 2023. Panic Recovery in Rust-based Embedded Systems. In *Proc. ACM PLOS*.
- [45] Zhiyao Ma and Lin Zhong. 2023. Bringing Segmented Stacks to Embedded Systems. In *Proc. ACM HotMobile*.
- [46] Aleksander Malinowski and Hao Yu. 2011. Comparison of embedded system design for industrial applications. *IEEE Trans. Industrial Informatics* (2011).
- [47] Anthony J Massa. 2002. *Embedded software development with eCos*. Prentice Hall Professional.
- [48] Ralph Moore. 2005. Link Service Routines. *Micro Digital* (2005).
- [49] Mohamed Abd El-Latif Mowad, Ahmed Fathy, Ahmed Hafez, et al. 2014. Smart home automated control system using android application and microcontroller. *Int. Journal of Scientific & Engineering Research* (2014).
- [50] Vikram Narayanan, Tianjiao Huang, David Detweiler, Dan Appel, Zhaofeng Li, Gerd Zellweger, and Anton Burtsev. 2020. RedLeaf: Isolation and Communication in a Safe Operating System. In *Proc. USENIX OSDI*.
- [51] Pimoroni. 2024. PMW3901. <https://github.com/pimoroni/pmw3901-python/tree/main>. Version 1.0.0. Accessed: 2025-04-24.
- [52] Pololu. 2022. V5311x. <https://github.com/pololu/v5311x-arduino>. Version 1.3.1. Accessed: 2025-04-24.
- [53] LLVM Project. 2023. Implementation of C++ ABI Exception Handling Level 1. <https://github.com/llvm/llvm-project/blob/f42482def236999b0f7896c09cd714b708861c8b/libunwind/src/UnwindLevel1.c>. Accessed: 2025-04-24.
- [54] RISC-V. 2024. *The RISC-V Instruction Set Manual: Volume II (Version 20240411)*. [https://riscv.org/technical/specifications/Chapter 3.7: Physical Memory Protection](https://riscv.org/technical/specifications/Chapter%203.7:Physical%20Memory%20Protection). Accessed: 2025-04-24.
- [55] Rust on Embedded Devices Working Group et al. [n. d.]. The Embedded Rust Book. <https://docs.rust-embedded.org/book/>. Accessed: 2025-04-24.
- [56] Alice Ryhl. 2023. Rewriting Binder Driver in Rust. <https://lore.kernel.org/rust-for-linux/20231101-rust-binder-v1-0-08ba9197f637@google.com/>. Accessed: 2025-04-24.
- [57] Friedrich Schon, Wolfgang Schroder-Preikschat, Olaf Spinczyk, and Ute Spinczyk. 2000. On interrupt-transparent synchronization in an embedded object-oriented operating system. In *Proc. IEEE Int. Symp. Object-Oriented Real-Time Distributed Computing (ISORC)*.
- [58] Wolfgang Schröder-Preikschat. 1994. *The logical design of parallel operating systems*. Prentice-Hall, Inc.

- [59] STMicroelectronics. 2020. RM0402 (Rev.6) Reference manual for STM32F412 advanced Arm-based 32-bit MCUs. Chapter 3.4.2: Adaptive real-time memory accelerator (ART Accelerator).
- [60] STMicroelectronics. 2024. RM0090 (Rev.21) Reference manual for STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 advanced Arm-based 32-bit MCUs. https://www.st.com/resource/en/reference_manual/rm0090-stm32f405415-stm32f407417-stm32f427437-and-stm32f429439-advanced-armbased-32bit-mcus-stmicroelectronics.pdf Chapter 30.3.3: Receiver - Overrun Error. Accessed: 2025-04-24.
- [61] STMicroelectronics. 2024. STM32CubeF0 HAL driver MCU component. <https://github.com/STMicroelectronics/STM32CubeF0/releases/tag/v1.11.5>. Version 1.11.5. Accessed: 2025-04-24.
- [62] STMicroelectronics. 2024. STM32CubeF4 HAL driver MCU component. https://github.com/STMicroelectronics/stm32f4xx_hal_driver/releases/tag/v1.8.3. Version 1.8.3. Accessed: 2025-04-24.
- [63] Rust Tools Team. 2021. svd2rust. <https://github.com/rust-embedded/svd2rust>
- [64] Theseus Developers. 2023. Support for unwinding the call stack and cleaning up stack frames. <https://github.com/theseus-os/Theseus/blob/562a39cf6c662738f7718473f6bbc010970dce53/kernel/unwind/src/lib.rs>. Accessed: 2025-04-24.
- [65] Vishakha D Vaidya and Pinki Vishwakarma. 2018. A comparative analysis on smart home system to control, monitor and secure home, based on technologies like gsm, iot, bluetooth and pic microcontroller with zigbee modulation. In *Proc. IEEE Int. Conf. Smart City and Emerging Technology (ICSCET)*.
- [66] Mathijs van de Nes and Joshua Barretto. 2023. Spin-based synchronization primitives. <https://crates.io/crates/spin>. Version 0.9.8. Accessed: 2025-04-24.
- [67] Rob Von Behren, Jeremy Condit, Feng Zhou, George C Necula, and Eric Brewer. 2003. Capriccio: scalable threads for internet services. *ACM SIGOPS Operating Systems Review* (2003).